

Semana 6

Tarea 4

Programar el backup automatico con Cron

Objetivo:

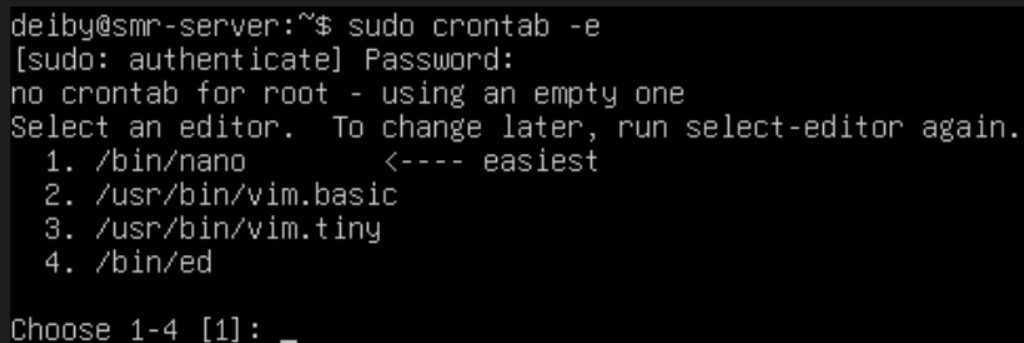
El objetivo de esta tarea es automatizar la ejecución del script de backup para que se realice de forma periódica sin intervención manual, garantizando la continuidad de las copias de seguridad.

1. Acceso al crontab

Se accedió al programador de tareas del sistema utilizando el siguiente comando:

```
- sudo crontab -e
```

Este comando permite editar las tareas programadas del usuario root, lo que garantiza que el script tenga permisos necesarios para ejecutarse correctamente.

A screenshot of a terminal window showing the execution of the 'sudo crontab -e' command. The prompt is 'deiby@smr-server:~\$'. The output shows a password prompt, confirmation that no crontab exists for root, and a list of editors to choose from: nano, vim.basic, vim.tiny, and ed. The user has selected nano.

```
deiby@smr-server:~$ sudo crontab -e
[sudo: authenticate] Password:
no crontab for root - using an empty one
Select an editor. To change later, run select-editor again.
 1. /bin/nano          <---- easiest
 2. /usr/bin/vim.basic
 3. /usr/bin/vim.tiny
 4. /bin/ed
Choose 1-4 [1]: _
```

2. Configuración de la tarea programada

Se añadió la siguiente línea al final del archivo:

```
- 0 2 * * * /usr/local/bin/backup_samba.sh
```

Esta configuración indica que el script de backup se ejecutará todos los días a las 02.00.

La estructura del cron se interpreta de la siguiente manera:

- 0 = minuto
- 2 = hora
- * = todos los días del mes
- * = todos los meses
- * = todos los días de la semana

```
GNU nano 8.4 /tmp/
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow  command
0 2 * * * /usr/local/bin/backup_samba.sh_
```

3. Verificación de la tarea programada

Para confirmar que la tarea se guardó correctamente, se utilizó el siguiente comando:

- `sudo crontab -l`

Este comando muestra las tareas programadas activas.

```
deiby@smr-server:~$ sudo crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow  command
0 2 * * * /usr/local/bin/backup_samba.sh
deiby@smr-server:~$
```

4. Resultado

Se ha configurado correctamente una tarea automática que ejecuta diariamente el script de backup, permitiendo mantener copias de seguridad actualizadas sin intervención manual.

Este sistema mejora la seguridad y disponibilidad de los datos en el servidor.